# How to impose variables within WRF

For certain studies, imposing variables can be a vital tool to test the relationship of physical mechanisms within the model. While there might be multiple ways to accomplish this task, the following steps have been thoroughly tested and proven to work.

1. **Introduction to input files**
    a. The easiest way to impose variables is to have **matching netCDF4** files compared to your desired output files in both **time** and **resolution**.
        i. I.e., if you are running a simulation with the goal of having 64 hours of output data in 1 hour intervals across 2 domains, you need 64 files per domain spaced an hour apart and matching the internal time components of your data (if your starting timestep is 2025-07-05_10:00:00, your input files should match this)
    b. WRF recognizes files as **<name>.d<domain>_<date>**. To create successful input files that WRF can dynamically use, it's important to **rename only the <name> component** in the file name as the name is more arbitrary. For example, if you want to input temperature, you could have a list of files such as **tempinput.d02_2025-07-05_10:00:00.** The most straightforward way to do this is to rename previous output data, which is typically formatted as **wrfout_d01_2007-08-17_21:00:00.** Therefore, the only changes to make in bulk are with the name and changing the underscore to a dot.
        i. It is unproven if changing from the underscore to the dot is extremely critical, but testing has shown it is favorable.
    c. In short, WRF utilizes Run-Time I/O Option to coordinate input and output "streams". Clicking on the link will provide more of an overview of how this works. WRF has dedicated stream numbers for specific versions of the model as well as critical processes like the met.em files. We have found that the **auxinput7** stream is the most suitable for imposing as it is unused by any preexisting WRF processes.
2. **Critical Namelist Settings**
    a. To properly setup input streams, the following lines should be added underneath &time_control:

        **auxinput7_inname='name.d<domain>_<date>'**
        **auxinput7_interval=60,60**
        **frames_per_auxinput7=1, 1**
        **io_form_auxinput7=2**

        i. **auxinput7_inname** determines the file structure WRF will look for. The name should be replaced with whatever you choose, as referenced in section 1b. **auxinput7_interval** is the **minute interval** between files for each domain, so for hourly output, 60 is the necessary option. **frames_per_auxinput7** indicates how many timesteps are in each file. Using raw output files typically results in 1 timestep per file, though this approach *might* work with a singular stitched file, but that is untested.
3. **Changing Internal Variables**
    a. It's often good practice to input **new** variables rather than overwriting existing variables through input streams. As such, it might be important to change variable names within your input data to ensure there are new variables. An example script of how to do this is listed below. Another possible step to take is renaming times within files to ensure they match up with your new simulation.

Running a command such as the following: **ncap2 -O -s 'Times[Time,DateStrLen]="2007-08-17_21:00:00"' name.d01_2007-08-17_21:00:00 name.d01_2007-08-17_21:00:00** works to change one file. To run in bulk often depends on the data you're dealing with, however, another script below is an approach to rename the internal Times variable to match the file title while ensuring no files are permanently deleted or damaged.

4. **Code changes within WRF**
    a. In order for WRF to use the desired variables within your input files, changes must be made internally to ensure input variables are used within your simulation.
    b. The first change is to update the **WRF/Registry/Registry.EM_COMMON** file to include the input variables.
        i. The most straightforward way to do this is to duplicate a line existing already and modify as such:
            1. state real LH ij misc 1 - rh "LH" "LATENT HEAT FLUX AT THE SURFACE" "W m-2"
            2. state real LH_INPUT ij misc 1 - i7rh "LH_INPUT" "INPUT LATENT HEAT FLUX AT THE SURFACE" "W m-2"
        ii. Notice how the only variables changed are the **name** (**both <Sym> and <DNAME>), the description, and the I/O stream**.
            1. **<u>The I/O must be changed to i7rh if using auxinput7.</u>**
    c. All other changes made depend solely on the desired outcome. One example of this is with imposing surface fluxes. Given the task of replacing surface fluxes, let's say 'LH' with 'LH_INPUT', you want the model to calculate the fluxes first and then identify where those fluxes are then passed on to the rest of the model for further calculations. For this example, **WRF/phys/module_surface_driver.F** is the physics file where this occurs *(As a side tangent, .F files are the correct files to edit–not .f90 or .mod*). Generally, when adding in a new variable, the most straightforward way to ensure your variable is used is to copy exactly where partner variables are initialized within the code file.
        i. For example, an initial subroutine "surface driver" calls all necessary variables from the registry. Immediately after 'LH' appears, 'LH_INPUT" is added. Fortran lacks case sensitivity with local variables, but it's still best to mimic exactly how the model was already calling it.
        ii. From there, continue to identify all areas where relevant partner variables are initialized/referenced and copy for new variables as needed. Then, you are on the path to coding within your physics module to adapt, edit, or remove variables as fit!
    d. After making changes to code, especially within physics modules, ***you must recompile for changes to occur. It is essential to create a backup of your namelist file to ensure settings aren't deleted***.

5. **Executing Simulation**
    a. After all code changes have been made and WRF has been recompiled, you are ready to run your new WRF simulation!
    b. Good to test a few time steps first to ensure the model runs correctly with your code changes in addition to desired effects correctly working. In the event of a failure, refer to the WRF workflow document or repeat steps in this document to ensure no steps were missed.

# Referenced Scripts

rename.sh - example of how to rename

***must execute module load NCO first***

```
#!/bin/bash

# Directory containing the NetCDF files and namelist.input. Replace with yours
DIRECTORY="WRF/run"

# Navigate to the directory
cd "$DIRECTORY" || exit

# Rename the variable HFX to HFX_input in all files starting with fluxinput
for file in fluxinput*; do
 ncrename -v HFX,HFX_INPUT "$file"
 ncrename -v QFX,QFX_INPUT "$file"
 ncrename -v LH,LH_INPUT "$file"
done

echo "Renaming completed"
```

timerename.sh - an example script to set Times variable to match filename if using renamed WRF output.

***ensure there are backups of all files in case this doesn't work for your data. This file is not tested as thoroughly so while it worked for me, it doesn't guarantee it will work for you without fail.***

```
echo "Setting Times variable to match filename timestamp..."

for file in fluxinput.d*; do
   [ -e "$file" ] || continue

   echo "Processing $file..."

   new_time=$(echo "$file" | grep -oP '\d{4}-\d{2}-\d{2}_\d{2}:\d{2}:\d{2}')

   if [[ -z "$new_time" ]]; then
       echo "❌ Could not parse timestamp from filename $file"
       continue
   fi
```

```
    tmpfile=$(mktemp)
    cp "$file" "$tmpfile"

    # Properly shape the Times variable (Time, DateStrLen)
    ncap2 -O -s "Times[Time,DateStrLen]=\"$new_time\"" "$tmpfile" "$tmpfile" >/dev/null
2>&1

    if [[ $? -eq 0 ]]; then
        mv "$tmpfile" "$file"
        echo "Set Times(Time,DateStrLen) in $file to $new_time"
    else
        echo "Failed to update Times in $file"
        rm -f "$tmpfile"
    fi
done
```